

TRABAJO DE FIN DE GRADO

Ecosistema de Domótica Local Impulsado por Agentes de IA

Luxe Core AI

Autor: Jesús Fernández López

Titulación: Grado en Ing. Informática

Tutor: Dr. Rafael Muñoz Salinas



Universidad de Córdoba — EPSC

Córdoba, junio de 2026

► Índice

Introducción

Problema y Objetivos

Restricciones y Metodología

Especificación del Sistema

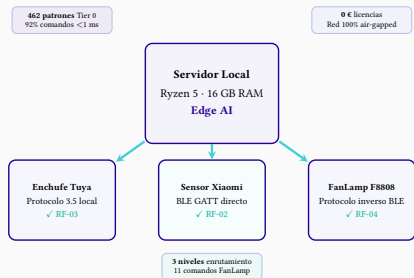
Integración de Hardware

Validación y Pruebas

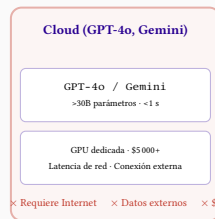
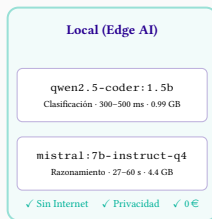
Demostración

Conclusiones

► Resumen Ejecutivo y Motivación



Modelos pequeños en CPU vs grandes en GPU



¿Qué es Luxe Core AI?

Sistema 100 % local de control de confort térmico orquestado por IA en el *edge*. **462 patrones** Tier 0 (92 %, <1 ms) · **0 €** licencias · Red air-gapped.

Principio rector

La tecnología del hogar no debería exigir ceder la privacidad a cambio de comodidad.

► El Problema de la Dependencia Cloud

Dependencia de conectividad

Sin Internet, el hogar inteligente se vuelve «inerte». Afecta a calefacción, ventilación y funciones esenciales.

Exposición de datos sensibles

Patrones de comportamiento viajan a servidores de terceros. Caso real: empleados escuchaban grabaciones de Alexa (Bloomberg, 2019).

Latencia y fragmentación

«Apaga la luz» tarda segundos. Tuya, Zigbee, BLE, LIN: protocolos incompatibles por diseño. Ecosistemas cerrados = barreras de salida.

Raíz común

La delegación de la inteligencia del hogar a infraestructuras externas.

La respuesta de Luxe Core AI

Capa de software que **entiende y coordina** hardware heterogéneo.

Dispositivos que no interoperan... interoperan.

Interoperabilidad = capa de orquestación inteligente, no estándares cerrados.

Ejemplo real: Enchufe Tuya → humidificador 24V + sensor Xiaomi + FanLamp → un solo sistema de confort, sin modificar hardware.

► Objetivo General del Proyecto

Objetivo general

Diseñar, implementar y validar un sistema de monitorización y control de confort térmico que opere de forma autónoma en una red local privada, eliminando cualquier dependencia de servicios en la nube.

Principios de diseño transversales:

Coste cero

0 € en licencias,
APIs externas
o suscripciones

Reproducibilidad

Mismo hardware
→ replicar
con la documentación

Hardware real

Pruebas solo sobre
dispositivos físicos,
no simulación

► Objetivos Específicos e Indicadores

1. Arquitectura de microservicios local✓

OpenClaw systemd · Interfaces desacopladas · Diagramas de despliegue

2. Control HVAC por bus LIN~ Pendiente

ESP32 + LINTTL3 · Firmware C++20 · Comunicación bidireccional

3. Integración IoT heterogénea✓ 3/3

Xiaomi BLE (RF-02) · Tuya local (RF-03) · FanLamp (RF-04)

Cobertura

92%

comandos

Latencia

<2 s

red local

Disponibilidad

99%

24 h sin caídas

Aislamiento

100%

air-gapped

► Restricciones y Factores de Diseño

Restricción temporal y económica

- Un curso académico → MVP con funcionalidades troncales
- Autor: estudiante sin ingresos — presupuesto **56,50 €**
- Ingeniería inversa como alternativa a la inversión

Impacto positivo

- Obliga a desarrollar competencias transferibles
- GATT Xiaomi sin autenticación: una herramienta cara lo habría ocultado
- Domótica inteligente $\neq$ grandes inversiones

Evolución de infraestructura

Fase 1: servidores UCO con GPU (prototipado)

Fase 2: torre Ryzen 5, 16 GB RAM, 100 % local

► Arquitectura de Red Air-Gapped

Problema: los dispositivos comerciales intentan abrir canales hacia sus nubes.

Solución: Router sin salida WAN

- TP-Link TL-MR6400 sin tarjeta SIM
- Actúa como *switch* Ethernet + AP WLAN
- Segmento 192.168.1.0/24 completamente aislado
- IPs estáticas — aislamiento físico verificable

Hotspot Spoofing (Tuya)

1. Punto de acceso móvil replica SSID
2. Emparejamiento contra nube Tuya
3. Extracción de *Local Key*
4. Se apaga el *hotspot*
5. Dispositivo migra al segmento air-gapped

Local Key nunca abandona el servidor.

► Stack Polígota y Justificación

Principio

La naturaleza dual de la domótica con IA (tiempo real + flexibilidad cognitiva) hace inviable un sistema monolítico en un solo lenguaje.

Python 3.12

Drivers de dispositivos

tinytuya, bleak

Enrutador de modelos

HTTP server, comfort advisors

→ Ecosistema maduro, prototipado rápido

Node.js 24

Orquestador OpenClaw

Canales Telegram y dashboard web

Interfaz de agentes IA

exec → drivers Python

→ Gestión nativa de canales

C++20

Firmware ESP32

Control determinista de memoria

Bus LIN (pendiente)

→ *Hard real-time* cuando se necesita

Cada lenguaje en su capa óptima, sin acoplar tiempos de ejecución ni forzar compromisos.

► Metodología: Docs-as-Code y Git

Control de versiones

- Repositorio privado GitHub
- Cada contribución auditada
- Trazabilidad: decisión de diseño $\leftrightarrow$ código
- Base para futura iniciativa empresarial

Docs as Code

- Memoria $\text{\LaTeX}$ + código en mismo repo
- Consistencia tipográfica profesional
- 9 ADR (*Architecture Decision Records*)
- Diagramas TikZ versionados

Uso de IA en el desarrollo

- **OpenCode**: asistente de codificación (software libre)
- **DeepSeek V4**: modelo de inferencia vía API
- **Toda línea revisada y validada por el autor**

Herramientas de gestión

SSH con llaves RSA, Cockpit (puerto 9090), systemd.

► Arquitectura de 3 Capas

Componentes desacoplados:

1. Servidor de orquestación

Torre Ryzen 5, 16 GB RAM, Ubuntu Server 24.04 headless

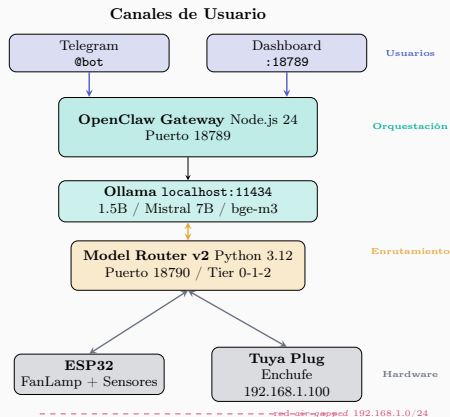
- OpenClaw Gateway (Node.js 24, puerto 18789)
- Ollama (puerto 11434): qwen2.5 1.5B + mistral 7B + bge-m3
- Drivers Python (Tuya, BLE, FanLamp)
- Sensor Daemon + Model Router

2. Portátil de desarrollo

SSH + Git + Arduino CLI + $\LaTeX$

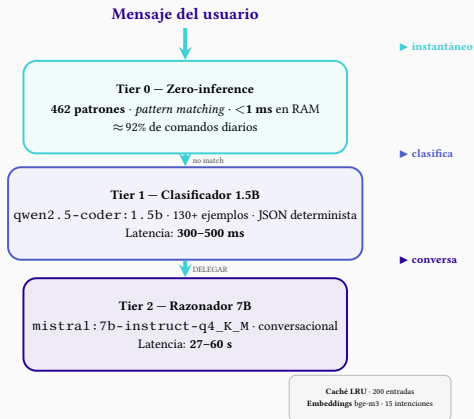
3. Nodo firmware ESP32

Puente BLE para FanLamp (futuro: LIN HVAC)



Comunicación exclusiva sobre 192.168.1.0/24. Ningún dato abandona el perímetro local.

► Model Router v2: Enrutamiento en 3 Niveles



Ejemplos

Com.	Niv.	Lat.
«vel.3»	T0	1,5 s
«casa»	T0	<1 ms
«enciendeT1	T1	300 ms
«buenos días»	T2	27–60 s

Circuit Breaker

Si el 1.5B falla, deriva al Tier 2.

► Comparativa de Modelos Evaluados

Proceso iterativo de selección

Tres fases de evaluación empírica sobre Ryzen 5 (16 GB RAM, sin GPU). Corpus de 500 comandos reales.

Modelo	Tarea	Latencia	Fallos	RAM
Qwen2.5 1.5B	Clasificación	0,3–0,5 s	0 %	0,99 GB
Qwen2.5 7B	Razonamiento	5–30 s	0 %	6,0 GB
Qwen3.5 4B	Razonamiento	45–180 s	12 %	3,8 GB
Qwen3.5 9B	Razonamiento	180–720 s	18 %	7,1 GB
Mistral 7B Q4	Razonamiento	27–60 s	0 %	4,4 GB

Qwen3.5: descartado

Modo *thinking* forzado → 0,6–1,2 tok/s en CPU.
3–12 min por consulta. Diseñado para GPU.

Arquitectura mixta final

Tier 1: Qwen2.5 1.5B (determinista)
Tier 2: Mistral 7B Q4 (conversacional)
Cada modelo especializado en su nivel.

► Asesores de Confort Térmico

Comfort Advisor (ASHRAE 55)

- Índice de calor NOAA (Rothfus)
- 6 zonas de confort adaptadas al clima andaluz
- Evaluación de humedad (seco / óptimo / húmedo)
- Detección de tendencia ($^{\circ}\text{C}/\text{hora}$)

Zonas térmicas:

1. Fresco ($< 23^{\circ}\text{C}$) — sin acción
2. Confortable ($23\text{--}26^{\circ}\text{C}$)
3. Cálido ($26\text{--}28^{\circ}\text{C}$) — ventilador 1
4. Caluroso ($28\text{--}31^{\circ}\text{C}$) — ventilador 3
5. Muy caluroso ($31\text{--}34^{\circ}\text{C}$) — ventilador 4
6. Extremo ($> 34^{\circ}\text{C}$) — ventilador 5, AC urgente

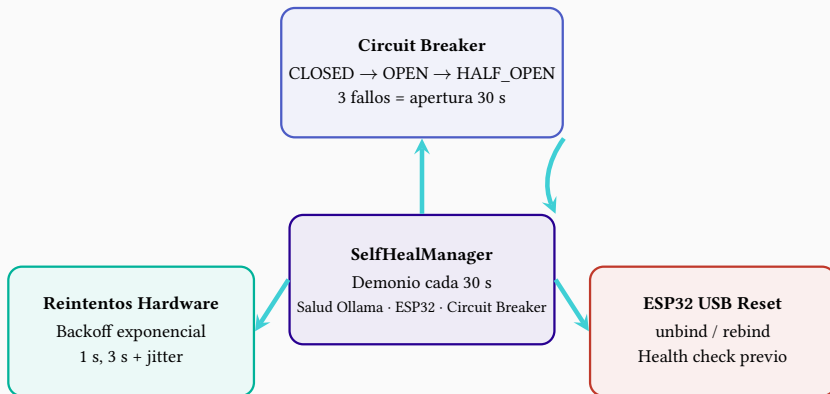
Smart Advisor (contexto exterior)

- Comparación interior vs. exterior (wttr.in)
- Lógica ventana/persiana (dif. térmica, UV, lluvia)
- Detección de presencia (manual/BLE/IP Wi-Fi)
- ***Human in the Loop***: preguntar antes de actuar

Auto-acciones condicionales:

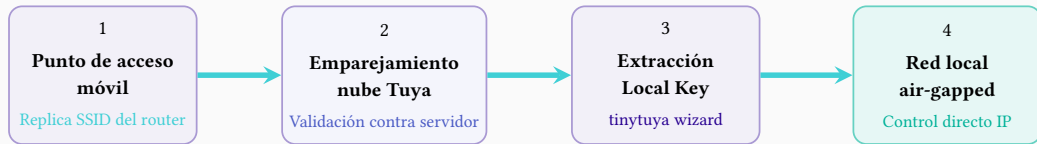
- Misma zona $\rightarrow$ no actuar
- +1 zona $\rightarrow$ preguntar
- +2 zonas $\rightarrow$ actuar automático
- Zona extrema $\rightarrow$ actuar siempre
- Sin presencia $\rightarrow$ nunca actuar

► Auto-Recuperación y Tolerancia a Fallos



Circuit Breaker (3 estados) · **Reintentos** (backoff + jitter) · **SelfHealManager** (30 s) · **ESP32 USB reset**

► Integración Tuya: Control Local y Hotspot Spoofing



Local Key nunca abandona el servidor – Control 100% local sin depender de Tuya



Protocolo tinytuya 3.5

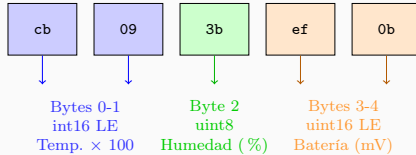
Local Key en servidor · IP 192.168.1.100

`status() · toggle() · set_state()`

Respuesta <2 s — validado en red air-gapped

► Sensores BLE Xiaomi: Lectura GATT Directa

Payload GATT (5 bytes): cb 09 3b ef 0b



0x09cb = 2507

0x3b = 59

0x0bef = 3055

25.07°C

59 % HR

3055 mV



Xiaomi LYWSD03MMC

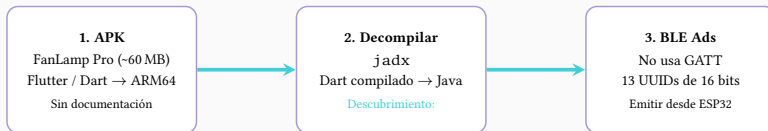
Lectura GATT directa

ebe0ccc1 (servicio ebe0ccb0)

cb093bef0b → **25,07°C** **59 % HR**
3055 mV

Sin *bind key* · Sin nube Xiaomi

► FanLamp F8808: Ingeniería Inversa del Protocolo



11 comandos verificados

OFF
Apagado total

FAN
ON / OFF

LIGHT
ON / OFF

1-5
Velocidades

NIGHT
Modo noche



Descubrimiento clave

APK FanLamp Pro decompilado (jadx, ~60 MB)

No usa conexiones GATT, sino anuncios BLE

Cada comando = 13 UUIDs de 16 bits · Emitidos desde ESP32

► Evolución del Controlador: Puente Serie ESP32

Fase 1 — BLE directo (heredado)

- `fanlamp_bt.py` → `btmgmt` (bluez)
- Requiere `sudo` (parar/reiniciar `bluetoothd`)
- Monopoliza el adaptador BT del servidor
- ~3 s por comando, inestable

Fase 2 — Puente ESP32 (actual)

- `fanlamp_control.py` → USB serie (115200 baud)
- ESP32 como radio BLE externa dedicada
- Sin `sudo` (grupo `dialout`)
- ~1,5 s por comando, reset DTR automático
- Adaptador BT del servidor libre para Xiaomi

Ventaja clave

PC → USB serie → ESP32 → BLE → FanLamp

El ESP32 que hoy es puente BLE podrá ser puente LIN para la HVAC. **Hardware reutilizado, coste 0 € adicional.**

► Suite de Pruebas: 500 Comandos

Pruebas realizadas (hardware real):

1. **Tuya:** conectividad IP, ON/OFF, alternancia → ✓
2. **BLE Xiaomi:** descubrimiento, GATT, decodificación → ✓
3. **FanLamp:** 11 comandos verificados físicamente → ✓
4. **Sensor Daemon:** lectura cada 60 s, JSON atómico → ✓
5. **Model Router:** 3 niveles validados → ✓
6. **Circuit Breaker:** fallo simulado + recuperación → ✓

Cobertura Tier 0 (500 comandos):

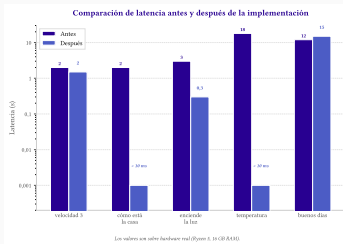
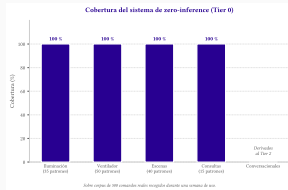
- Iluminación (35 variantes): 100 %
- Ventilador (50 variantes): 100 %
- Escenas (40 variantes): 100 %
- Consultas estado (15 variantes): 100 %
- Conversacionales → Tier 2: esperado

Suite automatizada

`tests/test_integration.py`
5 subsistemas verificados de forma coordinada. No destructiva: solo comandos de lectura.

► Resultados: Latencia y Cobertura

Escenario	Antes	Ahora	Modelo
«velocidad 3»	~2 s	~1,5 s	Tier 0
«cómo está la casa»	~2 s	0–1 ms	Tier 0
«me enciendes la luz»	~3 s	~300 ms	1.5B
«buenos días»	~12 s	27–60 s	Mistral 7B
«temperatura»	18 s	0–1 ms	Daemon



Disponibilidad

model-router sensor-daemon
openclaw-gateway

Operativos 24 h sin caídas.

► Demostración: Test de Integración Completa

Panel multimedia



VÍDEO DE DEMOSTRACIÓN

captura_demo.png + video_demo.mp4

Protocolo de Contingencia

Respaldo 1 — Local

► Abrir archivo local

video_demo.mp4

Respaldo 2 — YouTube

► Ver en YouTube

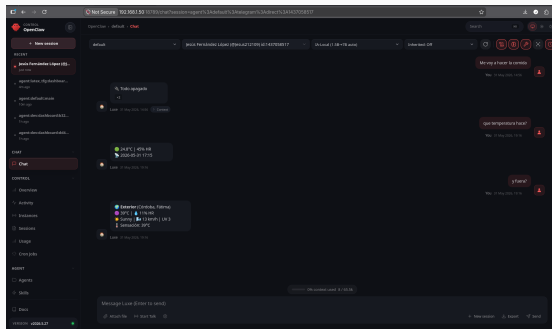
youtu.be/LuxeCoreAI



► Demostración Práctica: Telegram + Dashboard



Conversación real con Luxe Core AI vía Telegram



Dashboard web de OpenClaw (<http://192.168.1.50:18789>)

Comandos en lenguaje natural

→ interpretados en local

→ ejecución en hardware real.

Ningún mensaje abandona la subred.

► Logros Alcanzados vs Rúbrica

Verificados sobre hardware real

- **Arquitectura 3 capas escalable**
- **Control local Tuya (RF-03)** — ON/OFF <2 s
- **Lectura BLE Xiaomi (RF-02)** — GATT sin nube
- **FanLamp F8808 (RF-04)** — 11 comandos
- **Model Router v2** — 462 patrones, 92 % cobertura
- **SelfHealManager** — 24 h sin caídas
- **Canales Telegram + Dashboard** operativos

Estado de los módulos

Módulo	Estado	Obs.
Tuya Plug	✓	Air-gapped
BLE Xiaomi	✓	GATT directo
FanLamp	✓	11 comandos
OpenClaw	✓	Puerto 18789
Model Router	✓	3 niveles
ESP32 HVAC	Pendiente	Firmware C++20

5/6 módulos. 0 € en licencias.

► Trabajo Futuro: Corto Plazo

Firmware HVAC (ESP32 + LIN)

Prioridad inmediata — Hardware adquirido. esphome-fujitsu-halcyon como base. **2-3 semanas**

Eliminar puente ESP32

Control BLE directo con bleak. Elimina punto único de fallo. **1-2 semanas**

Streaming SSE Tier 2

Mostrar palabras conforme se generan. **1-2 semanas**

Bucle de control con OpenClaw

Conectar sensores BLE, Tuya y FanLamp al orquestador. **1-2 semanas**

Containerización Docker

docker-compose para replicabilidad total. **1-2 semanas**

Total corto plazo

5 líneas. Estimación: **5-10 semanas.**

► Trabajo Futuro: Medio y Largo Plazo

Medio plazo (meses)

- **Expansión *dashboard*:**
Analíticas energéticas, histórico ASHRAE 55 — 1–2 meses
- **Expansión sensorial:**
Sensor presencia LD2410, micrófono/altavoz — 1–2 meses
- **Base de datos vectorial:**
Qdrant o ChromaDB para recuperación semántica — 2–3 meses
- **Memoria conversacional:**
Embeddings con bge-m3, referencias implícitas — 2–3 meses

Largo plazo (investigación)

- **Aprendizaje adaptativo:**
Reinforcement learning para hábitos del usuario — varios meses
- **Nuevos protocolos:**
Zigbee, Z-Wave → más dispositivos — varios meses
- **Re-evaluación de modelos:**
Qwen3.5 (sin *thinking*), Llama 4, DeepSeek — continuo
- ***Fine-tuning* clasificador:**
Ajuste con datos reales de uso doméstico — varios meses

► Privacidad por Diseño: Reflexión

El principio fundamental

La tecnología del hogar no debería exigir ceder la privacidad a cambio de comodidad.

Lo que hemos comprobado:

- Es posible tener **ambas cosas** con hardware de consumo
- La arquitectura *air-gapped* es viable y verificable
- Los modelos pequeños en local **funcionan**
- La interoperabilidad no depende de la nube: depende del software

Más allá de lo técnico

La decisión de mantener todo en local fue una cuestión de principios.

Y en pocos años...

Modelos ligeros con capacidad cercana a los grandes cloud:
«¿se puede en local?» → «¿por qué en la nube?»



Luxe Core AI

Muchas gracias por su atención.

Quedo a disposición del ilustre tribunal
para resolver las cuestiones que consideren oportunas.

Jesús Fernández López
Grado en Ing. Informática — UCO
Junio 2026

